SMART TELECOM BILL ADVISOR

MongoDB with PyMongo and Machine Learning

Project Report

Project implementation	Single Python source file: **main.py**
Database	MongoDB Atlas
Python database library	PyMongo
Dataset size	7,043 records and 21 original columns
Machine learning	Decision Tree Classifier and Decision Tree Regressor

Student Name: _____
Registration Number: _____
Course / Section: _____
Submitted To: _____

1. Introduction

Telecommunication companies collect customer information such as service subscriptions, contract type, tenure, payment method and monthly charges. Analysing this information can help understand customer billing patterns. This project demonstrates how MongoDB can store customer records and how Python can retrieve those records to perform machine-learning tasks.

The Smart Telecom Bill Advisor performs two tasks: it classifies a customer into a Low, Medium or High monthly bill category, and estimates the customer's accumulated TotalCharges using service-related information. The complete executable workflow is written in one Python file named **main.py**.

2. Project Objectives

• Connect a Python application to MongoDB Atlas using PyMongo.
• Import and store telecom customer records in a MongoDB collection.
• Retrieve MongoDB documents and prepare them with Pandas.
• Train a Decision Tree classifier for monthly bill category prediction.
• Train a Decision Tree regressor for accumulated TotalCharges estimation.
• Evaluate the models using Accuracy, Precision, Recall, F1 Score, R^2 Score and MAE.

3. Dataset Description

The project uses the Telco Customer Churn dataset supplied with the project files. The CSV contains **7,043 records** and **21 original columns**. Each row represents one telecom customer. Example attributes include customerID, gender, SeniorCitizen, Partner, Dependents, tenure, PhoneService, InternetService, Contract, MonthlyCharges, TotalCharges and Churn.

The dataset is commonly distributed as the IBM Telco Customer Churn dataset. Dataset source page: Kaggle – Telco Customer Churn. The supplied CSV is included in the ZIP under data/telecom_data.csv.

4. MongoDB and PyMongo Implementation

MongoDB Atlas is used as the cloud database. PyMongo is the official Python driver used by the application to communicate with MongoDB. The database is named **telecom_advisor** and the collection is named **telecom_records**.

The main.py program reads the MONGO_URI setting from a local .env file, creates a MongoClient, selects the database and collection, reads the CSV using Pandas, converts TotalCharges and MonthlyCharges to numeric values, and inserts the records. It creates indexes on customerID and MonthlyCharges. The program then retrieves the documents using find() and converts them into a Pandas DataFrame.

Important: For a clean repeatable import, the script clears only the project collection telecom_advisor.telecom_records before inserting the CSV again. Do not use that collection for unrelated data. The .env file must contain the user's own Atlas connection URI and must not be shared or submitted with a real password.

5. Data Preprocessing

The application converts MonthlyCharges and TotalCharges to numeric data types. Invalid or blank values in TotalCharges are converted to missing values. Missing values are handled before model training. Categorical columns are transformed into numeric indicator columns using Pandas get_dummies(). For classification, missing feature values are filled with zero. For regression, missing numeric feature values are filled using the training dataset's feature medians, with remaining missing values filled with zero.

For classification, BillCategory is generated from MonthlyCharges using pandas.qcut() with three quantile groups labelled Low, Medium and High. These are data-derived groups and are not official telecom tariff bands.

MonthlyCharges is excluded from the classifier input because it is used to create the target category.

6. Model Implementation

6.1 Decision Tree Classifier

The classifier predicts the BillCategory target. The customer identifier, Churn, MonthlyCharges, TotalCharges and BillCategory are excluded from the input feature set. The remaining customer service attributes are encoded into numeric columns. The data is divided into 80% training and 20% testing data using train_test_split(), random_state=42 and stratify=y. A DecisionTreeClassifier with max_depth=5 is trained using fit() and evaluated using predictions on the test set.

6.2 Decision Tree Regressor

The regression model predicts TotalCharges, which represents accumulated charges in the supplied dataset rather than a guaranteed next invoice. Rows without a valid TotalCharges value are removed. The target column, customerID, Churn and the classification-only BillCategory are excluded from the regression inputs. The remaining features are one-hot encoded and missing numeric values are imputed with their medians. An 80/20 train-test split is used. A DecisionTreeRegressor with max_depth=10 and min_samples_leaf=5 is trained. Decision-tree regression predicts averages of training target values in terminal leaves, which avoids the negative extrapolation seen in the earlier Linear Regression version for this positive target.

7. Evaluation Results

The following values were obtained by running the same preprocessing and model settings locally on the supplied 7,043-row dataset with an 80/20 train-test split and random_state=42. The results printed by the student's own run should be treated as the final submission values if the local package versions or dataset differ.

Task	Metric	Result
Classification – Decision Tree	Accuracy	0.8935 (89.35%)
Classification – Decision Tree	Weighted Precision	0.9015 (90.15%)
Classification – Decision Tree	Weighted Recall	0.8935 (89.35%)
Classification – Decision Tree	Weighted F1 Score	0.8949 (89.49%)
Regression – Decision Tree	R^2 Score	0.9978
Regression – Decision Tree	Mean Absolute Error (MAE)	71.5177

Metric interpretation: Accuracy measures overall classification correctness. Precision measures how many predicted class assignments were correct. Recall measures how many actual class records were identified. F1 Score is the harmonic mean of precision and recall. R^2 measures the proportion of target variation explained by the regression model. MAE is the average absolute prediction error in the target's units. Higher is generally better for Accuracy, Precision, Recall, F1 and R^2; lower is better for MAE.

8. Conclusion

This project demonstrates an end-to-end integration of MongoDB Atlas, PyMongo, Pandas and machine learning in a single Python source file. The application imports 7,043 telecom customer records into MongoDB, retrieves and preprocesses the data, trains a Decision Tree classifier and a Decision Tree regressor, and evaluates them using all six required metrics. The project provides a practical example of storing NoSQL documents and using the stored data for predictive analysis.

9. How to Run the Project

1. Extract the ZIP and open its folder in VS Code.
2. Create a virtual environment: `py -m venv venv`
3. Activate it: `.\venv\Scripts\Activate.ps1`
4. Install packages: `pip install -r requirements.txt`
5. Copy .env.example to .env and add the MongoDB Atlas URI.
6. Run the complete application: `python main.py`.

The project requires an internet connection to reach MongoDB Atlas. Keep the real .env file private. The ZIP intentionally contains only .env.example, not database credentials.

10. Project File Structure

Smart_Telecom_Bill_Advisor_SingleFile/
■■■ main.py (complete executable Python code)
■■■ data/telecom_data.csv (dataset)
■■■ Project_Report.pdf (this report)
■■■ requirements.txt
■■■ .env.example
■■■ README.md